LAB | Relational and NoSQL Database Fundamentals

Part 1 — Relational Databases and NoSQL

Q. 1 — Build a Small Relational Model

Q. 2 — Insert and Join the Relational Data

```
CREATE database week2_db;

use week2_db;

# creating Relational table:

CREATE TABLE customers
(
    customer_id VARCHAR(50) PRIMARY KEY,
    customer_name VARCHAR(50),
    city VARCHAR(50)

);
CREATE TABLE orders
(
    order_id VARCHAR(50) PRIMARY KEY,
    customer_id varchar(50) NOT NULL,
    amount_gbp DECIMAL(10,2),
    status VARCHAR(20) NOT NULL,
    FOREIGN KEY(customer_id)
    REFERENCES customers(customer_id)
);

# Insert values into customers and orders:
```

```sql
INSERT INTO customers
(customer_id, customer_name, city)
VALUES
(1, 'Amelia Clarke', 'London'),
(2, 'Oliver Bennett', 'Manchester'),
(3, 'Aisha Khan', 'Leeds');

INSERT INTO orders
(order_id, customer_id, amount_gbp, status)
VALUES
(101, 1, 45.50, 'complete'),
(102, 2, 18.00, 'cancelled'),
(103, 1, 62.25, 'complete'),
(104, 3, 30.00, 'complete');

# Validate:
SELECT * FROM customers;
SELECT * FROM orders;

# Write a join that shows:
/*
order_id
customer_name
city
amount_gbp
status
*/

SELECT o.order_id, c.customer_name, c.city, o.amount_gbp, o.s
tatus
FROM orders as o
INNER JOIN customers as c
WHERE o.customer_id= c.customer_id;
```

Q. 3 — Represent the Same Customer as a Flexible Document

Use a Python dictionary to represent this customer:



customer_id = 1
name = Amelia Clarke
city = London
email = amelia@example.com
preferences = newsletter: True, language: en

Create a second customer without an `email` field.

Explain what this shows about flexible document structure.

```
cust_info1={ 'customer_id': 1,
                        'name': 'Amelia Clarke',
                        'city': 'London',
                        'email': 'amelia@example.com',
                        'preferences': {

'newsletter': True,

'language': 'en'
                                                        }
                        }


cust_info2={ 'customer_id': 1,
                        'name': 'Amelia Clarke',
                        'city': 'London',
                        'preferences': {
```

```
    'newsletter': True,
    'language': 'en'
                                                    }
                            }
```

Document-oriented flexible databases allows to have:

1. flexible records
2. attributes vary according to the records
3. nested data is allowed naturally
4. one application usually reads most of the document together instead of entities as separate tables in a traditional relational model.

Q. 4 — MCQ: Relational Database

Which requirement strongly supports a relational design?

Solution: . C. Orders must reference valid customers and transaction consistency is important.

Part 2 — Document, Key-Value, and Column-Family Designs

Q. 5 — Choose the Better NoSQL Model

Match each requirement to **document**, **key-value**, or **column-family**.



A. Product records have different optional attributes.
B. Session ID must return session data very quickly.
C. Large event data is grouped by a known key and may contain many columns.
D. Customer profile contains nested addresses and preferences.

A. document

B. column-family

C. key-value

D. document

Q. 6 — Build a Key-Value Example

Use a Python dictionary as a simple key-value representation.

Store:

> 💡 SESSION-1001 → customer 1, cart_count 3
> SESSION-1002 → customer 2, cart_count 1
> SESSION-1003 → customer 3, cart_count 5

Look up `SESSION-1002`.

```
sessions_dict= {
                                        'session-1001': {'custome
r': 1,
                                        'cart_count': 3}

                                        'session-1002': {'custome
r': 2,
                                        'cart_count': 1}

                                        'session-1003': {'custome
r': 3,
                                        'cart_count': 5}
}

print(sessions_dict['session-1002'])
```

Q. 7 — Design Three Product Documents

Create three JSON-like product documents.

Use:

💡 Product 1:
product_id = P101
name = Office Chair
colour = Black
material = Mesh
Product 2:
product_id = P102
name = Laptop Stand
colour = Silver
height_adjustable = True
Product 3:
product_id = P103
name = Notebook Set
page_count = 200
paper_size = A5

Explain why forcing every product to contain every possible field could be awkward.

Solution:

Product 1:

```
{ 'product_id': 'P101',
    'name': 'Office Chair',
    'color': 'Black',
    'material': 'Mesh'
}
```

Product 2:

```
{ 'product_id': 'P102',
    'name': 'Laptop Stand',
```

```
    'color': 'Silver',
    'height_adjustable ': 'True'
}
```

Product 3:

```
{ 'product_id': 'P103',
    'name': 'Notebook Set',
    'page': 200,
    'paper_size': 'A5'
}
```

Each of the products belong to the same category `Products` but has different attributes. Document databases provide flexibility to store the varying attributes across the `Products` category.

Forcing every product to contain every possible field could be awkward because not every attribute belongs to all the product categories.

Q. 8 — MCQ: Key-Value Database

Which access pattern is the best fit for a key-value database?

Solution: A. Find a session directly from its session ID.

Part 3 — Graph Data, ACID, and BASE

Q. 9 — Model Connected Data

BritMart is checking possible fraud links.

Use these nodes:



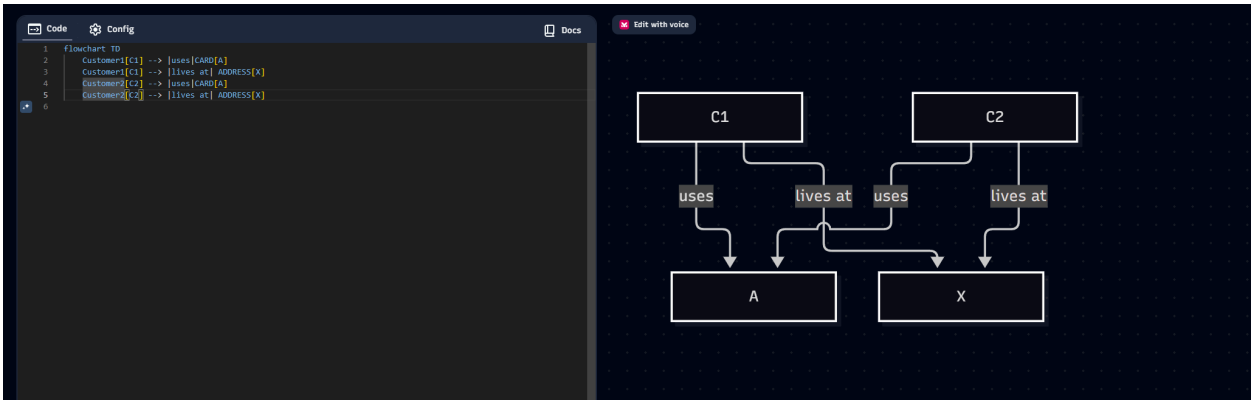
Customer C1
Customer C2
Card CARD-A
Address ADDR-X

Relationships:



C1 uses CARD-A
C2 uses CARD-A
C1 lives_at ADDR-X
C2 lives_at ADDR-X

Draw the relationships using text arrows or Mermaid.



Q. 10 — Identify the ACID Property

Match each example to **Atomicity**, **Consistency**, **Isolation**, or **Durability**.



A. An order and its payment should not be half-completed.
B. A successful commit remains stored after a restart.
C. Two transactions should not see unsafe partial work from each other.
D. A transaction must leave the database in a valid state.

Solution:

A. Atomicity

B. Durability

C. Isolation

D. Consistency

Q. 11 — ACID or BASE?

Choose the stronger fit:

Requirement	ACID or BASE?	Reason
Bank-like payment update	ACID	Data should not be partial; have consistency; cannot interfere with other transactions; should be stored after a successful commit. Main focus is transaction correctness
Social activity count can be briefly out of date	BASE	Designed to keep responding to requests and will eventually become consistent; main focus is availability
Order and stock update must remain consistent	ACID	Transaction correctness
Very high availability is more important than immediate consistency	BASE	Availability and distributed convergence; Data stored in the distributed system may change over time as the system synchronizes but the system stays available.

Q. 12 — MCQ: BASE

Which statement best describes a BASE-style design?

Solution: B. Temporary inconsistency may be accepted to support availability and scale

Part 4 — Fixed vs Flexible Schema and Choosing a Database Model

Q. 13 — Fixed or Flexible Schema?

Choose the better starting approach.

Data	Fixed or flexible?	Reason
Payment transaction with required fields	Fixed	data consistency
Product catalogue with category-specific attributes	Flexible	each product might have different attributes

Data	Fixed or flexible?	Reason
Customer order line with required quantity and price	Fixed	data consistency
Event payload where optional fields may change	Flexible	different events might have different fields which can vary

Q. 14 — Choose Relational or NoSQL

For each requirement, choose a starting model and explain why.



1. Customer orders with strong foreign-key relationships.
2. Product descriptions with many optional attributes.
3. User session lookup by session ID.
4. Fraud investigation based on many connected entities.

1. Relational model with fixed schema
2. Document: optional attributes can vary across products
3. key-value: user session values are stored under unique session ids
4. graph: data can be represented here as nodes and relationships

Q. 15 — Review a Poor Design Choice

BritMart stores payment transactions as flexible documents with no required fields.

Some records contain:

`payment_id`

but no:

`order_id`

`amount_gbp`

`status`

Explain two risks and suggest a better starting design based only on today's concepts.

Solution:

Data involving transactions are generally represented as a relational model, where each entity has a relationship with the other. A row with only payment_id and no other information in incorrect, since it does not follow **ACID** consistency model.

1. Garbage value being ingested from a source and stored
2. no relationship between orders and payments can create difficulty in understanding the data

Q. 16 — MCQ: Choosing a Database

BritMart must store orders, customers, and payments with strong relationships and transaction correctness. Which is the strongest starting choice?

Solution: D. Relational database

Optional Practice

Q. 17 — Design the Same Product Two Ways

Model this product as:

1. a relational table row;
2. a document.



product_id = P200
name = Smart Desk
price_gbp = 450
colour = White
motor_count = 2
height_min_cm = 70
height_max_cm = 120

Then add this second product:

product_id = P201
name = Notebook
price_gbp = 5
page_count = 200
paper_size = A5

Solution:

1. relational model: It is not a good practice to store data with varying column in a relational model while may give back unnecessary null values.
2. Document data provides more flexibility.

```
product= {
    {
                    ‘product_id’: ‘P200’,
                    ‘name’: ‘Smart Desk’,
                    ‘price_gbp’: 450,
                    ‘colour’: ‘White’,
                    ‘motor_count’: 2,
                    'height_min_cm': 70,
                    'height_max_cm': 170
                    },
                    {
                    'product_id': 'P201',
                    'name': 'Notebook',
                    'price_gbp': 5,
                    'page_count': 200,
                    'paper_size': 500
                    }
    }
```

Q. 18 — Find Shared Fraud Links

Use this data:

💡 relationships = [
("C1", "CARD-A"),
("C2", "CARD-A"),
("C1", "ADDR-X"),
("C2", "ADDR-X"),
("C3", "CARD-B"),
]

Find the values shared by both `C1` and `C2`.

Do this with Python lists, sets, and loops already covered in Week 1.

Solution: Both C1 and C2 shares the same card and address.

```
relationships = [
("C1", "CARD-A"),
("C2", "CARD-A"),
("C1", "ADDR-X"),
("C2", "ADDR-X"),
("C3", "CARD-B"),
]

c1_seen_data= set()
c2_seen_data= set()

for idx, val in enumerate(relationships):
  if val[0] == 'C1':
    c1_seen_data.add(val[1])

  if val[0] == 'C2':
    c2_seen_data.add(val[1])

print(c1_seen_data)
print(c2_seen_data)
```

```
shared_val= c1_seen_data.intersection(c2_seen_data)

print(shared_val)
```

Q. 19 — Database Choice Review

BritMart has these four systems:

> 💡 checkout orders
> product catalogue
> website sessions
> fraud relationship investigation

For each one, record:

> 💡 database model
> schema style
> consistency need
> main access pattern

Solution:

checkout orders:

database model: relational

schema style: fixed

consistency: ACID

main access pattern: transactions and joins

product catalogue:

database model: document

schema style: flexible

consistency: ACID

main access pattern: product document

website sessions:

database model: key-value

schema style: flexible

consistency: BASE

main access pattern: session keys

fraud relationship investigation:

database model: graph

schema style: flexible

consistency: ACID/BASE

main access pattern: nodes and relationships

Q. 20 — Explain the Trade-Off

A team says:

> *"NoSQL is always faster, so all BritMart data should move to NoSQL."*

Write a short response using only today's concepts.

Solution: NoSQL has high read and write throughput, however it also depends on the type of workload, as an example if the company has data for orders, shipping information then a relational model is used with fixed schema having ACID consistency, whereas other datasets such as product catalog or session logs comes under NoSQL, where relational tables do not fit naturally.